

NutriTrack Cloud

Documentation Technique et Guide de Déploiement

Projet	CLD2 — Cloud Computing, par Ricardo Cardoso
Stack	Vue.js + Node.js + MySQL
Hébergement	Vercel + Render + TiDB Cloud + Cloudinary
Coût	0 CHF/mois (100% gratuit)

Table des matières

1. Présentation du projet
2. Architecture technique
3. Technologies utilisées
4. Fonctionnement du code
5. Les modèles Cloud (IaaS, PaaS, SaaS)
6. Comparatif des services Cloud
7. Guide de déploiement
8. Conclusion

1. Présentation du projet

NutriTrack Cloud est une application web de suivi nutritionnel permettant aux utilisateurs de gérer leur alimentation quotidienne. Ce projet a été développé dans le cadre du cours CLD2 (Cloud Computing) avec pour objectif d'apprendre à déployer une application full-stack sur des services cloud gratuits.

Fonctionnalités principales

- Authentification sécurisée (inscription, connexion, JWT)
- Profil utilisateur avec objectifs personnalisés (calories, macros)
- Journal alimentaire par repas (petit-déjeuner, déjeuner, dîner, collation)
- Suivi du poids avec graphique d'évolution
- Statistiques visuelles (graphiques calories et macros sur 7/30/90 jours)
- Gestion des aliments avec upload d'images

2. Architecture technique

L'application suit une architecture classique en 3 tiers : Frontend, Backend et Base de données. Chaque composant est hébergé sur un service cloud distinct, tous gratuits.

Schéma d'architecture

Composant	Technologie	Hébergement	Rôle
Frontend	Vue.js 3 + Vite	Vercel	Interface utilisateur (SPA)
Backend	Node.js + Express	Render	API REST
Base de données	MySQL	TiDB Cloud	Stockage des données
Images	CDN	Cloudinary	Stockage et optimisation des images

Flux de communication

Le frontend et le backend communiquent via des requêtes HTTP (API REST). Chaque requête du frontend vers le backend inclut un token JWT dans le header Authorization pour authentifier l'utilisateur.

- L'utilisateur effectue une action (ex: ajouter un aliment)
- Le frontend envoie une requête HTTP (POST /api/food) avec le token JWT
- Le backend vérifie le token, traite la requête, interroge la BDD
- Le backend retourne une réponse JSON
- Le frontend met à jour l'interface

3. Technologies utilisées

Frontend

- **Vue.js 3** — Framework JavaScript réactif pour construire l'interface utilisateur
- **Vite** — Outil de build ultra-rapide pour le développement
- **Vue Router** — Gestion des routes côté client (SPA)
- **Chart.js + vue-chartjs** — Création des graphiques (poids, calories, macros)
- **Tailwind CSS** — Framework CSS utilitaire pour le design

Backend

- **Node.js** — Environnement d'exécution JavaScript côté serveur
- **Express** — Framework web minimaliste pour créer l'API REST
- **JWT (jsonwebtoken)** — Génération et vérification des tokens d'authentification
- **bcrypt** — Hashage sécurisé des mots de passe
- **Multer** — Middleware pour gérer les uploads de fichiers (images)
- **mysql2** — Driver MySQL pour Node.js avec support des Promises
- **Cloudinary SDK** — Upload et gestion des images dans le cloud

Pourquoi ces choix ?

Express : Simplifie la création d'une API REST. Sans Express, il faudrait gérer manuellement le routing, le parsing JSON, les headers HTTP, etc.

Multer : Nécessaire car les uploads de fichiers utilisent le format multipart/form-data qu'Express ne sait pas lire nativement. Multer parse ce format et rend le fichier accessible via req.file.

bcrypt : Permet de hasher les mots de passe avant de les stocker. Si la base de données est compromise, les mots de passe restent protégés car le hash est irréversible.

JWT : Permet une authentification stateless. Le token contient l'ID utilisateur signé, évitant de stocker les sessions côté serveur.

4. Fonctionnement du code

Structure du projet

Le projet est divisé en deux dossiers principaux : backend/ et frontend/. Chaque partie a son propre package.json et peut être déployée indépendamment.

Point d'entrée Backend (backend/src/index.js)

Ce fichier initialise le serveur Express, configure les middlewares (CORS, JSON parsing), et monte les différentes routes de l'API.

Les routes de l'API

Route	Méthode	Description
/api/auth/register	POST	Inscription d'un utilisateur
/api/auth/login	POST	Connexion (retourne un JWT)
/api/profile	GET/PUT	Récupérer/modifier le profil
/api/weight	GET/POST/DELETE	Gestion du suivi de poids
/api/food	GET/POST/PUT/DELETE	Gestion des aliments
/api/mealentry	GET/POST/PUT/DELETE	Journal alimentaire

Point d'entrée Frontend (frontend/src/main.js)

Ce fichier initialise l'application Vue.js, configure le router (Vue Router), et monte l'application sur l'élément #app du DOM.

Routing côté client

Vue Router gère la navigation côté client. Chaque route est associée à un composant Vue (view). Les routes protégées vérifient la présence d'un token JWT dans le localStorage.

5. Les modèles Cloud (IaaS, PaaS, SaaS)

Le cloud computing propose trois modèles de services principaux, qui diffèrent par le niveau de gestion laissé au client versus au fournisseur.

IaaS — Infrastructure as a Service

Le fournisseur gère l'infrastructure physique (serveurs, réseau, stockage). Le client gère tout le reste : système d'exploitation, runtime, application.

- Exemples : AWS EC2, Google Compute Engine, DigitalOcean Droplets
- Avantages : Contrôle total, très flexible
- Inconvénients : Maintenance importante, expertise requise

PaaS — Platform as a Service

Le fournisseur gère l'infrastructure ET la plateforme (OS, runtime, serveur web). Le client gère uniquement son application.

- Exemples : Render, Vercel, Heroku, Railway
- Avantages : Déploiement simple (git push), pas de maintenance serveur
- Inconvénients : Moins de contrôle, parfois plus cher à grande échelle

SaaS — Software as a Service

Le fournisseur gère absolument tout. Le client est simplement utilisateur du logiciel.

- Exemples : Gmail, Slack, Notion, Netflix
- Avantages : Aucune maintenance, prêt à l'emploi
- Inconvénients : Aucune personnalisation possible

DBaaS — Database as a Service

Sous-catégorie de PaaS spécialisée pour les bases de données. Le fournisseur gère l'installation, les sauvegardes, les mises à jour et la haute disponibilité.

- Exemples : TiDB Cloud, MongoDB Atlas, Supabase

CDN — Content Delivery Network

Réseau de serveurs répartis mondialement qui mettent en cache le contenu statique (images, CSS, JS). L'utilisateur reçoit le contenu depuis le serveur le plus proche, réduisant la latence.

- Utilisé par : Vercel (frontend), Cloudinary (images)

Positionnement de NutriTrack

NutriTrack utilise principalement le modèle PaaS : on déploie le code, les plateformes gèrent le reste.

Service	Modèle	Justification
Vercel	PaaS	Déploiement automatique du frontend Vue.js
Render	PaaS	Exécution du backend Node.js
TiDB Cloud	DBaaS	Base MySQL managée
Cloudinary	SaaS	Service d'images prêt à l'emploi

6. Comparatif des services Cloud

Avant de choisir la stack finale, plusieurs alternatives ont été évaluées pour chaque composant.

Frontend — Hébergement statique

Service	Avantages	Inconvénients
Vercel ✓	Rapide, Git push, SSL auto, CDN mondial	100 déploiements/jour
Netlify	Forms gratuits, fonctions serverless	Build limité 300min/mois
Cloudflare Pages	Bandwidth illimité, très rapide	Interface moins intuitive
GitHub Pages	Simple, intégré à GitHub	Pas de server-side, repo public

Choix : Vercel — Déploiement instantané via Git, CDN mondial, SSL automatique, excellente intégration avec Vue.js.

Backend — Hébergement Node.js

Service	Avantages	Inconvénients
Render ✓	Simple, logs clairs, Git deploy	Cold start ~30s
Koyeb	Cold start court (~5s)	RAM limitée (256MB)
Fly.io	Pas de cold start, multi-régions	Configuration complexe
Railway	Interface moderne, MySQL inclus	Plus de free tier (5\$/mois)

Choix : Render — Simplicité d'utilisation, déploiement Git automatique. Le cold start de 30s est acceptable pour un projet d'école.

Base de données — MySQL

Service	Avantages	Inconvénients
TiDB Cloud ✓	Compatible MySQL, 5GB gratuit, serverless	Latence si région éloignée
Aiven	Vrai MySQL, bonne performance	Expire après 30j inactif
PlanetScale	Excellent service	Free tier supprimé en 2024

Choix : TiDB Cloud — Compatible MySQL (pas de migration nécessaire), 5GB de stockage gratuit suffisant pour un projet d'école.

Stockage images

Service	Avantages	Inconvénients
Cloudinary ✓	25GB, transformations auto, CDN	Limité en transformations/mois
ImageKit	Simple, bonne documentation	Moins connu
Firebase Storage	Google, fiable	Pas de transformations

Choix : Cloudinary — SDK Node.js facile à intégrer, CDN mondial pour des images rapides, 25GB gratuit largement suffisant.

7. Guide de déploiement

Étape 1 : Cloudinary (Images)

- Créer un compte sur cloudinary.com
- Sur le Dashboard, noter : Cloud Name, API Key, API Secret

Étape 2 : TiDB Cloud (Base de données)

- Créer un compte sur tidbcloud.com
- Créer un cluster Serverless (gratuit) en région Frankfurt
- Cliquer Connect > General et noter : Host, Port, User, Password
- Dans SQL Editor, exécuter le script de création des tables

Étape 3 : Render (Backend)

- Créer un compte sur render.com (connexion GitHub)
- New > Web Service, connecter le repo GitHub
- Configurer : Root Directory = backend, Build = npm install, Start = npm start
- Ajouter toutes les variables d'environnement (DB, JWT, Cloudinary)
- Noter l'URL générée (ex: <https://nutritrack-api.onrender.com>)

Étape 4 : Vercel (Frontend)

- Créer un compte sur vercel.com (connexion GitHub)
- Add New > Project, importer le repo
- Configurer : Root Directory = frontend
- Ajouter la variable VITE_API_URL avec l'URL Render
- Noter l'URL générée (ex: <https://mon-app.vercel.app>)

Étape 5 : Configurer CORS

Retourner sur Render > Environment et ajouter FRONTEND_URL avec l'URL Vercel

Variables d'environnement Backend

Variable	Exemple	Description
PORT	3000	Port du serveur
FRONTEND_URL	https://app.vercel.app	URL du frontend (CORS)
DB_HOST	gateway01...tidbcloud.com	Hôte TiDB Cloud
DB_PORT	4000	Port MySQL
DB_USER	xxxxxx	Utilisateur BDD
DB_PASSWORD	xxxxxx	Mot de passe BDD
DB_NAME	nutritrack	Nom de la base
DB_SSL	true	Activer SSL
JWT_SECRET	longue_chaine_secrete	Clé de signature JWT
CLOUDINARY_CLOUD_NAME	xxxxxx	Nom du cloud
CLOUDINARY_API_KEY	123456789	Clé API
CLOUDINARY_API_SECRET	xxxxxx	Secret API

8. Conclusion

Récapitulatif de la stack

Composant	Service	Coût
Frontend	Vercel	Gratuit
Backend	Render	Gratuit
Base de données	TiDB Cloud	Gratuit (5GB)
Images	Cloudinary	Gratuit (25GB)

Ce que j'ai appris

- Déployer une application full-stack sur des services cloud
- Comprendre les modèles IaaS, PaaS, SaaS et leurs différences
- Configurer des variables d'environnement pour séparer dev et production
- Gérer les problèmes de CORS entre frontend et backend
- Utiliser un CDN (Cloudinary) pour stocker et servir des images
- Connecter une base de données cloud avec SSL
- Mettre en place une authentification JWT sécurisée

Liens du projet

Application : <https://cld-2-nutritrack-cloud.vercel.app>

GitHub : https://github.com/Ric-Oli/CLD2_Nutritrack_Cloud